

il theorem prover aiuta a risolvere il famoso problema ...

$$\Gamma \models F \longrightarrow$$

Molto spesso si tenta di risolvere questo problema, per esempio per verificare che un sistema soddisfi i requisiti necessari, oppure per descrivere le PETI o PETRI (che vedrai in automatica...)

Abbiamo visto che queste formule possono essere risolti:

- Nella logica proposizionale con le tavole delle verità o per risoluzione
- Nella logica del primo ordine la cosa è un po' più complicata, perché la formula ricade nella famiglia delle formule **INDECIDIBILI** (vedrai più avanti...)

SPASS è un theorem prover molto famoso e utile sviluppato circa 20 anni fa dal Max Planck Institute, per i primi 5 anni non sembrava utile a causa della sua complessità computazionale. Intorno agli anni 2010 nasce il **TPTP WORLD**, un progetto per sviluppare una sintassi universale per i compilatori del t.p.

È anche utile il **TPTP PROBLEM LIBRARY**, problemi di cui si conosce la risposta, utili per verificare il corretto funzionamento e la performance del t.p.

Noi vedremo i t.p. per la logica del I ordine, ma esistono dimostratori che passano un dominio specifico (ESEMPIO: LIA)

la struttura di SPASS è un file di testo con un tasto "SUBMIT"

All'interno del file di testo va messa una serie di LISTE:

- DESCRIZIONE PROBLEMA *
- DESCRIZIONE SIMBOLI
- // FORMULE (ASSIOMI)
- // FORMULE (CONGETTURE)
- // SETTINGS SPASS

$\vdash \models F$

CONNETTIVO	IN SPASS
$\neg$	not
$\wedge$	and
$\vee$	or
$\rightarrow$	implies
$\leftrightarrow$	equiv
$\forall x (F)$	forall ([x], F)
$\exists x (F)$	exists ([x], F)
=	equal

se avessi
 $\forall x \forall y \dots$
 posso anche
 scrivere
 forall([x,y],...)

forall([x], F)

SPASS non è CASE SENSITIVE: volendo posso anche scrivere NOT, o EQUIV, o ancora EQUAL, e il software lo accetterà



ATTENZIONE:

Anche se non è importante al fine di risolvere i problemi, tutti i campi della lista DESCRIZIONE PROBLEMA devono essere compilati, altrimenti il codice non viene compilato

NOTA: se non si vuole scrivere niente, basta scrivere una cosa così per ogni campo...

name ($\{ \text{ * } \}$)

author ($\{ \text{ * } \}$)

e così via ...

All'interno della list-of-symbols si dichiarano i simboli che useremo nelle formule. Vediamo come si fa ...

SIMBOLI	IN SPASS
COSTANTI : a, b	Dopotutto, le costanti non sono altro che funzioni costanti (arity 0...) $\rightarrow \text{functions}[(a, 0), (b, 0), (f, 1), (g, 2)]$
LETTERE FUNZIONALI : f arity 1 g arity 2	Dopotutto, le variabili booleane non sono altro che lettere predicative di arity 0...
LETTERE PREDICATIVE : P arity 1 Q arity 2	$\text{predicates}[(P, 1), (Q, 2), (C, 0), (\Delta, 0)]$
VARIABILI BOOLEANE : C, D	IMPORTANTE: functions va dichiarato PRIMA di predicates, altrimenti ottergo un errore!

E le VARIABILI ?

Le variabili non vanno dichiarate, ma sono inserite
 e comprese automaticamente da SPASS grazie alle
 proprietà dei quantificatori (se io nella formula
 scrivo $\forall x, \dots$ SPASS decifra x
 come una variabile)

ATTENZIONE: SPASS e tutti gli altri theorem prover
 gestiscono esclusivamente **ENUNCIATI**, cioè FORMULE CHIUSE

Ma come si dichiarano le formule?

Le formule si distinguono in ASSIOMI e CONGETTURE
Entrambi i tipi si dichiarano così:

↓
⊢ ⊨ ⊢

formula (inserisci qui la formula).

← OCCHIO AL
PUNTO (è tipo il ";" in
alcuni linguaggi di
programmazione)

Non serve che la formula sia in forma normale
prenessa o skolemizzata: SPASS fa automaticamente
le trasformazioni necessarie per risolvere la formula,
basta che questa sia scritta bene!

Anche l'output è un file di testo.

Le prime righe indicano le CLAUSOLE della
formula: quelle a sinistra di "→" sono
negate, mentre quelle a destra sono positive

Le righe successive mostrano le regole usate per
provare a dedurre □

POSSIBILI RISULTATI DI SPASS:

proof found $\longrightarrow$ Ho trovato $\square$

completion found $\longrightarrow$ Non posso determinare $\square$

ran out of time $\longrightarrow$ Il server ha un limite massimo di tempo. Se non risolve la formula in

peggiore dei casi: $\longrightarrow$ questo tempo, ottengo
non so se $\Gamma \models F$ o questo output
no!

COME VERIFICARE SE UNA FORMULA E' UNA TAUTOLOGIA:

Devo verificare se $\models F$. Sappiamo che questo succede quando $\top F \vdash_R \square$, e quindi se ottengo proof found da SPASS. Se ottengo completion found, significa che $\top F$ è SODDISFACIBILE

① PROBLEMA 1 PUZZLE TLTCL

Problem : Dreadbury Mansion

Version : Especial.

Theorem formulation : Reduced > Complete.

English : Someone who lives in Dreadbury Mansion killed Aunt Agatha. Agatha, the butler, and Charles live in Dreadbury Mansion, and are the only people who live therein. A killer always hates his victim, and is never richer than his victim. Charles hates no one that Aunt Agatha hates. Agatha hates everyone except the butler. The butler hates everyone not richer than Aunt Agatha. The butler hates everyone Aunt Agatha hates. No one hates everyone. Agatha is not the butler. Therefore : Agatha killed herself.

Ci serviranno i seguenti ingredienti:

FUNCTIONS:

$(a, 0) \rightarrow \text{AGATHA}$

$(b, 0) \rightarrow \text{BUTLER}$

$(c, 0) \rightarrow \text{CHARLES}$

COSTANTI

PREDICATES:

$(K, 2) \quad K(x, y) = x \text{ ha ucciso } y$

$(H, 2) \quad H(x, y) = x \text{ odia } y$

$(R, 2) \quad R(x, y) = x \text{ è più ricco di } y$

$(L, 1) \quad L(x) = x \text{ vive nella Mansion}$

PREDICATI

ora costruiamo le formule ().

CONGETTURA: Agatha si è uccisa da sola

ASSIOMI:

hanno considerati tutti i seguenti punti:

- a, b e c vivono nella Mansion

- nessun altro vive nella Mansion

CIOE'...

Se x vive nella Mansion, allora $x = a$ oppure $x = b$ oppure $x = c$

- Se x uccide y , allora x odia y e x e' meno ricco di y

- Se Agatha odia x allora Charles non odia x

- Agatha odia tutti tranne il maggiordomo

CIOE'...

Se x non e' il maggiordomo, allora a odia x

- Il maggiordomo odia tutti quelli meno ricchi di Agatha

CIOE'...

Se x non e' più ricco di a , allora b odia x

- Se a odia x , allora b odia x

- Nessuno odia tutti

CIOE'

Per ogni x esiste y tale che x non odia y

- $a \neq b$

Dunque il codice finale e' il seguente

```
begin_problem(Pelletier54).
```

```
list_of_descriptions.  
name(*Puzzle fatto con SPASS*).  
author(*Sean Giachi*).  
status(unsatisfiable).  
description(*Esercizio fatto in classe con il professor Achille Frigeri. Problema preso dal set di problemi del TLTL*).  
end_of_list.
```

non
importante

```
list_of_symbols.  
functions[(a,0),(b,0),(c,0)].  
predicates[(K,2),(R,2),(H,2),(L,1)].  
end_of_list.
```

```
list_of_formulae(axioms).
```

```
%someone who is in the mansion killed Agatha  
formula(exists([x], and(K(x,a), L(x)))).
```

```
%Agatha, the butler, and Charles live in Dreadbury Mansion, and are the only people  
% who live therein  
formula(L(a)).  
formula(L(b)).  
formula(L(c)).  
formula(forall([x], implies(L(x), or(equal(x,a), equal(x,b), equal(x,c))))).
```

```
%A killer always hates his victim, and is never richer than his victim.  
formula(forall([x,y], implies(K(x,y), H(x,y)))).  
formula(forall([x,y], implies(K(x,y), not(R(x,y))))).
```

```
%Charles hates no one that Aunt Agatha hates.  
formula(forall([x], implies(H(a,x), not(H(c,x))))).
```

```
%Agatha hates everyone except the butler.  
formula(forall([x], implies(not(equal(x,b)), H(a,x)))).
```

```
%The butler hates everyone not richer than Aunt Agatha  
formula(forall([x], implies(not(R(x,a)), H(b,x)))).
```

```
%The butler hates everyone Aunt Agatha hates.  
formula(forall([x], implies(H(a, x), H(b, x)))).
```

```
%No one hates everyone.  
formula(forall([x], exists([y], not(H(x, y))))).
```

```
%Agatha is not the butler.  
formula(not(equal(a, b))).
```

```
end_of_list.
```

```
list_of_formulae(conjectures).
```

```
%Agatha killed herself.  
formula(K(a,a)).
```

```
end_of_list.
```

```
list_of_settings(SPASS).  
{*  
set_flag(PGiven,1).  
set_flag(PProblem,0).  
*}  
end_of_list.  
  
end_problem.
```

Per verificare se la soluzione è giusta, controlla
che ottieni "proof found" con la clausola della
congettura e "completion found" senza